RECIPE MANAGEMENT SYSTEM

PROJECT REPORT

Submitted in fulfilment for the Course Project of UBCA301L – Full Stack Application Development

in

B.C.A

by

Mohammed Azhan Palli (23BCA0288)

Mohammed Taha B (23BCA0021)

Mohammed Saihan J (23BCA0175)

Mohammed Musab S (23BCA0004)

Under the guidance of

Dr. Brindha.K

SCORE

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

November 2025

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

Fall Semester 2025-26

TABLE OF CONTENTS

ABSTRACT

The management of recipes across diverse categories and multiple users poses significant challenges due to the limitations of traditional storage methods, which often lack centralised access, collaboration, and efficient organisation. *RECIP* is a comprehensive recipe management system designed to overcome these issues by offering a unified platform for both administrators and users. It enables centralised recipe storage, allowing administrators to curate and manage collections while providing users with easy access to categorised recipes. The system ensures a seamless user experience through intuitive navigation, efficient search functionality, and enhanced recipe discovery. Additionally, RECIP integrates social engagement features such as liking, sharing, and user interactions, thereby fostering an active and connected community of cooking enthusiasts.

1. Problem Statement

In the current digital environment, managing recipes across multiple categories and users remains inefficient and fragmented. Traditional methods of recipe storage and sharing—such as handwritten notes, spreadsheets, or basic websites—lack centralised organisation, user collaboration, and accessibility. As a result, users face difficulties in discovering, saving, and sharing recipes, while administrators struggle to maintain and update large recipe collections effectively. There is a need for a centralised, user-friendly system that allows efficient recipe management, seamless user interaction, and community engagement through features like categorisation, search, and social sharing.

2. System design

The **RECIP** system is designed to provide an efficient, user-friendly, and scalable recipe management platform for both administrators and end-users. The system architecture follows a modular and layered design, ensuring smooth interaction between various components.

2.1 Architectural Overview

The system consists of three main layers:

- **Frontend Layer:**
    - Developed using modern web technologies such as React or Laravel Blade for intuitive user interaction.
    - Provides user-friendly interfaces for browsing, searching, adding, and managing recipes.

    - Ensures responsive design for accessibility on both desktop and mobile devices.

- **Backend Layer:**
    - Implemented using frameworks such as Laravel (PHP) or Node.js to handle business logic.
    - Manages user authentication, recipe storage, and category organisation.
    - Provides APIs for smooth communication between frontend and database.
- **Database Layer:**
    - Stores all recipe data, user information, and category details in a structured format.
    - Uses relational databases like MySQL or PostgreSQL for efficient data retrieval and management.

2.2 Key System Components

- **User Module:** Handles user registration, login, and profile management.
- **Recipe Module:** Allows adding, editing, deleting, and viewing recipes, organised by categories.
- **Admin Module:** Enables administrators to manage recipe collections, user roles, and system updates.
- **Search and Filter Module:** Provides quick access to recipes using keywords, ingredients, or categories.
- **Social Interaction Module:** Supports liking, sharing, and community engagement features.

2.3 Data Flow

1. Users interact with the frontend interface to search or upload recipes.
2. The frontend sends requests to the backend via RESTful APIs.
3. The backend processes the request and retrieves or stores data in the database.
4. The processed information is displayed to users through an intuitive UI.

2.4 Security and Performance

- Secure authentication using encrypted credentials.

- Data validation at both client and server sides.
- Optimised database queries to ensure fast response time and scalability.

3. Software Requirement Specifications

The Software Requirement Specification (SRS) defines the functional, non-functional, and environmental requirements of the **RECIP** system — a web-based recipe management platform built using **Next.js**, **Express.js**, and **MongoDB**. The purpose of this system is to allow administrators to manage recipes efficiently and users to explore, like, and discover new recipes seamlessly.

3.1 Functional Requirements

The functional requirements describe what the system is expected to perform.

3.1.1 User Management

- Users can **sign up**, **log in**, and **log out** securely.
- Authentication is handled via **NextAuth or JWT**, ensuring data privacy.
- Users can view all available recipes, search for specific ones, and access details.

3.1.2 Admin Management

- Admins can log in to a dedicated dashboard.
- Admins can **add**, **update**, **delete**, and **view** recipes.
- Admins can monitor the **total recipe count** and user interactions.
- Admins have control over recipe visibility and can manage categories.

3.1.3 Recipe Management

- Each recipe includes:
    - Recipe name
    - Description
    - Ingredients
    - Preparation steps

    - Category (e.g., Indian, Chinese, Mexican, etc.)
    - Optional image upload
- Recipes are stored in **MongoDB**, and uploaded images are stored in the **public/uploads** folder using **Node's fs module**.
- Recipes can be fetched, searched, or deleted dynamically through REST API routes.

3.1.4 File Upload Handling

- Images are uploaded via form-data and stored on the server using Express and the fs module.
- File paths are stored in the database and retrieved dynamically.
- The upload directory is auto-created if missing.

3.1.5 Recipe Interactions

- Users can **like or unlike** recipes.
- The number of likes is updated dynamically using the **PUT API route**.
- Users can **copy recipe links** for sharing with others.

3.1.6 Search and Filtering

- Users can search recipes by **name**, **ingredient**, or **category** using a dynamic search bar.
- Filtering options allow users to browse by specific cuisine types (e.g., Indian, Afghani, Chinese, etc.).

3.1.7 Recipe Display

- Recipes are displayed with title, description, category, and image cards.
- Each recipe card links to a detailed recipe page showing ingredients, steps, author name, and like count.

3.2 Non-Functional Requirements

These define the system's quality and operational constraints.

3.2.1 Performance Requirements

- The system should handle multiple concurrent users efficiently.

- The recipe fetching and rendering process must complete within **1 second** under normal load.
- File uploads should support images up to **2 MB** in size.

3.2.2 Security Requirements

- All API endpoints are protected from unauthorized access.
- Inputs are validated and sanitized to prevent **XSS** or **injection** attacks.
- Admin actions are restricted to authenticated users only.

3.2.3 Reliability

- Database hosted on **MongoDB Atlas** ensures consistent uptime.
- System automatically reconnects to MongoDB if the connection drops.

3.2.4 Scalability

- The app architecture supports future scaling — more categories, rating systems, or community features can be added easily.
- Next.js and Express.js APIs are modular, allowing for horizontal scaling.

3.2.5 Usability

- Simple and modern interface built using **Next.js** and **Tailwind CSS**.
- Mobile-responsive UI for all screen sizes.
- Search and filtering enhance recipe discoverability.

3.2.6 Maintainability

- Backend code follows **modular design** and **MVC principles**.
- Separate routes for POST, GET, PUT, and DELETE APIs.
- Clear logging messages for debugging (e.g., "Recipe added successfully ✅").

3.3 System Environment Requirements

Component	Technology / Tool	Purpose
Frontend	Next.js (React Framework)	Client interface and routing

Component	Technology / Tool	Purpose
Backend	Express.js (Node.js)	API handling and business logic
Database	MongoDB (via Mongoose)	Storage for recipes, users, and categories
Styling	Tailwind CSS	Responsive and modern UI design
Authentication	JWT / NextAuth	Secure user sessions
Hosting	Vercel (Frontend), Render/Railway (Backend)	Deployment
File Handling	Node fs module	Image upload and management
Version Control	Git & GitHub	Source code management

3.4 Constraints

- Requires a stable internet connection for API requests.
- Browser compatibility limited to Chrome, Edge, Firefox, and Safari.
- Image upload restricted to .jpg, .jpeg, and .png formats.

3.5 Assumptions

- Users have a valid email for registration.
- Admin credentials are predefined during setup.
- Recipes are added in supported categories.
- MongoDB and server are correctly configured before deployment.

4. Implementation

The implementation phase of the **RECIP Recipe Management System** involves converting the design and functional requirements into a fully operational web-based application. This project integrates **Next.js**, **Express.js**, and **MongoDB**, providing a seamless platform for users to explore and manage recipes, while allowing administrators to control recipe data efficiently.

The system was implemented using **Next.js** for the frontend, **Express.js** for backend routing and API handling, and **MongoDB** for database management. The structure ensures scalability, responsiveness, and a clean user experience.

The **frontend** is designed using **Next.js**, offering server-side rendering for optimized performance and SEO benefits. The user interface is developed with **Tailwind CSS**, ensuring modern, responsive layouts. Various pages such as the Home Page, Recipe Details Page, Add Recipe Page, and Admin Dashboard are implemented for both user and admin interactions. Features like search functionality, category filters, and recipe preview sections enhance the usability of the website. Data fetching between frontend and backend is handled using **Axios**, and React's state management ensures that updates (like adding or deleting recipes) are reflected instantly.

The **backend** is powered by **Express.js**, which serves as the core API layer connecting the frontend with the database. It includes routes for adding, deleting, updating, and fetching recipes. The RESTful API design allows clean separation between client and server logic. When the admin uploads a recipe, the image is processed and stored in the public/uploads directory using Node's fs module, while metadata such as recipe name, description, and image path are saved in the MongoDB database.

For **database management**, **MongoDB Atlas** is used as a cloud database service, providing secure and scalable storage. The schema is defined using **Mongoose**, making it easier to handle and validate data. The recipe collection includes fields such as recipe name, description, image, category, and like count. The like functionality is implemented by updating the recipe document in MongoDB whenever a user interacts with the like button. All CRUD (Create, Read, Update, Delete) operations are implemented efficiently to ensure smooth data flow between the client and server.

Integration between all components is achieved through REST API calls. Next.js sends HTTP requests to Express routes, which perform database operations and return results in JSON format. The admin dashboard dynamically updates based

on the recipe data retrieved from MongoDB. Authentication and access control are implemented to restrict certain actions (like adding or deleting recipes) only to administrators.

During the development phase, the system was tested using **Postman** for backend route validation and **browser-based testing** for frontend behavior. Console logging and structured error-handling were used to identify and fix issues quickly. The application was verified for responsiveness across devices to ensure a consistent user experience.

Finally, the application was deployed successfully. The **frontend (Next.js)** was hosted on **Vercel**, while the **backend (Express.js)** and **MongoDB** were hosted using **Render/Railway** and **MongoDB Atlas** respectively. Environment variables such as database connection strings and keys were securely managed using .env files.

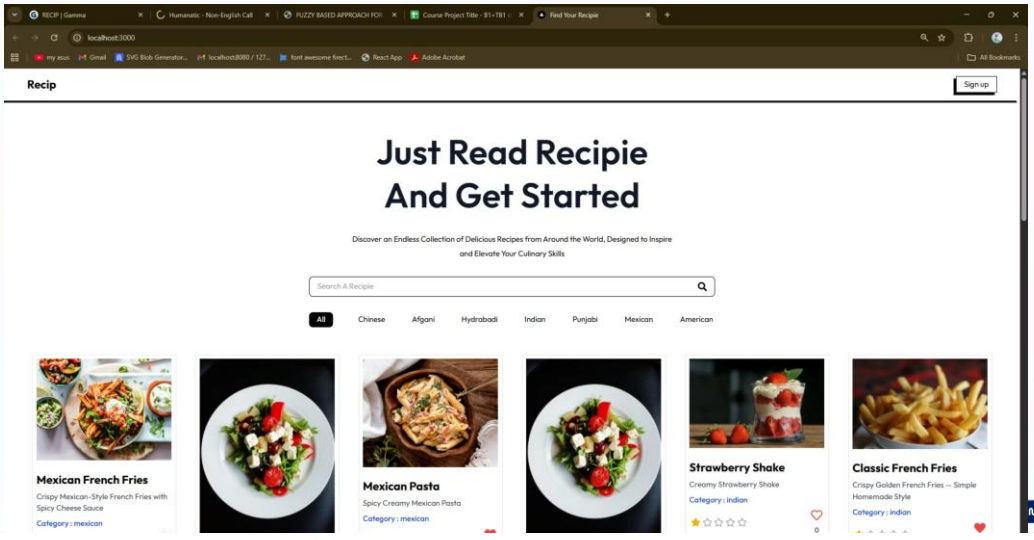
Overall, the implementation resulted in a **fully functional, user-friendly Recipe Management System** where users can view, search, and like recipes, and admins can manage the complete recipe collection efficiently. The integration of Next.js, Express.js, and MongoDB ensures that the application remains fast, secure, and scalable for future enhancements.

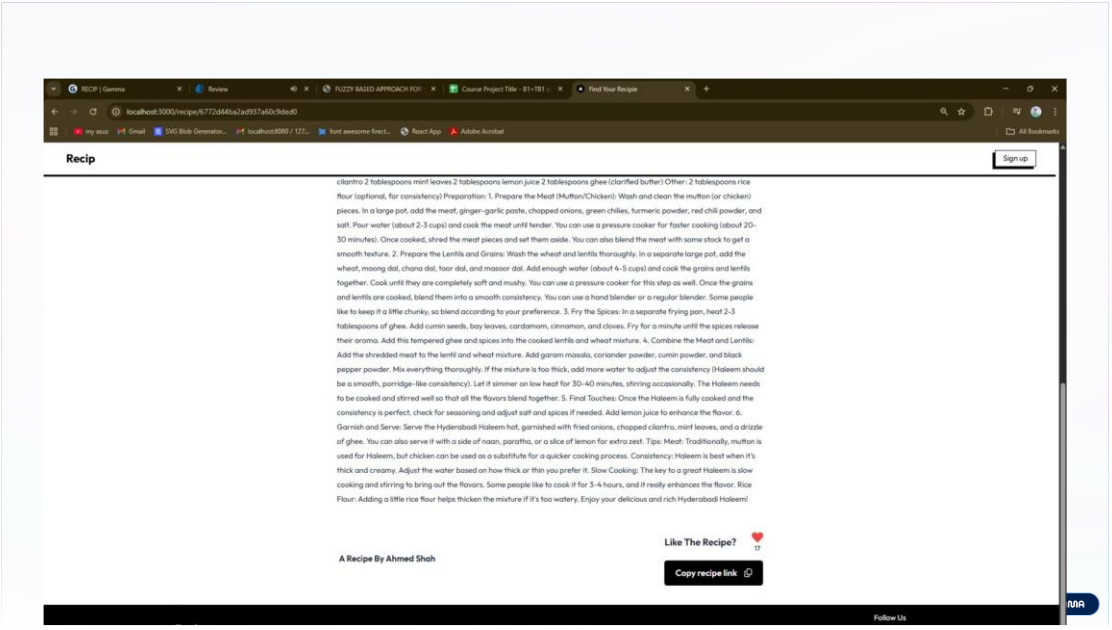
4.1 Source code

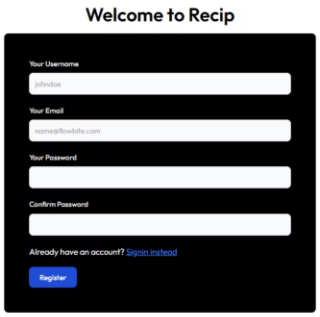
https://github.com/Mohammed-Azhan/recip

4.2 Output

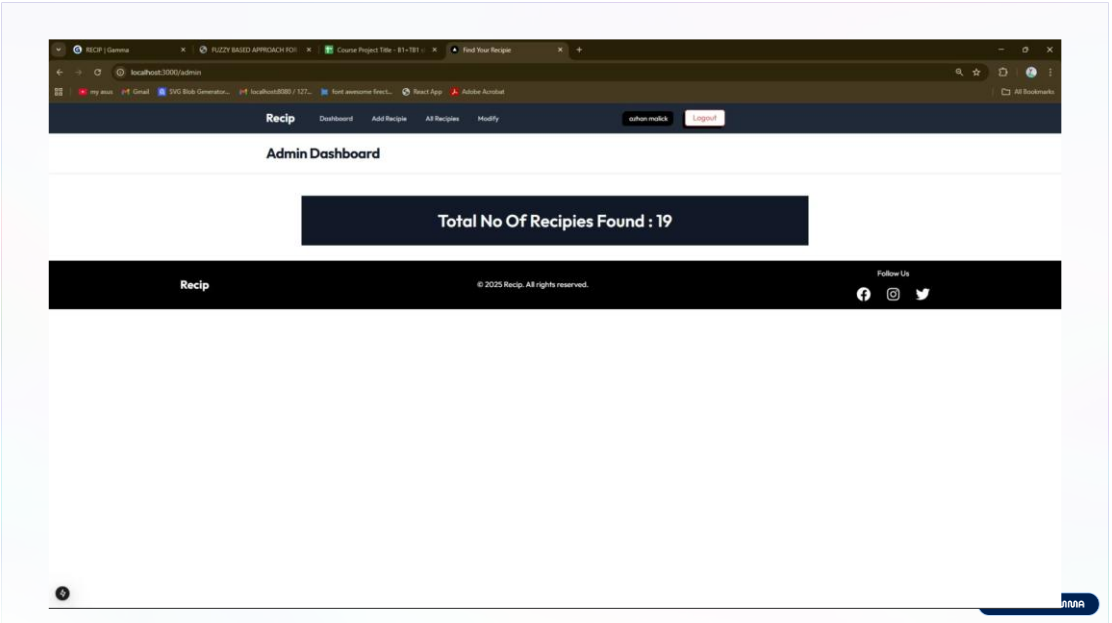
4.2 Screenshots

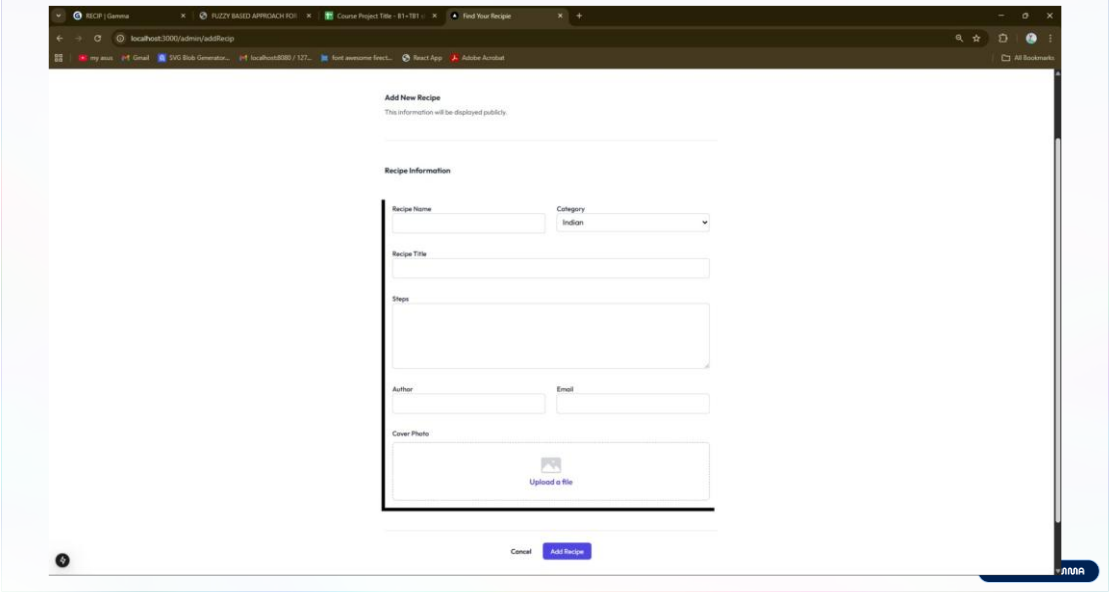


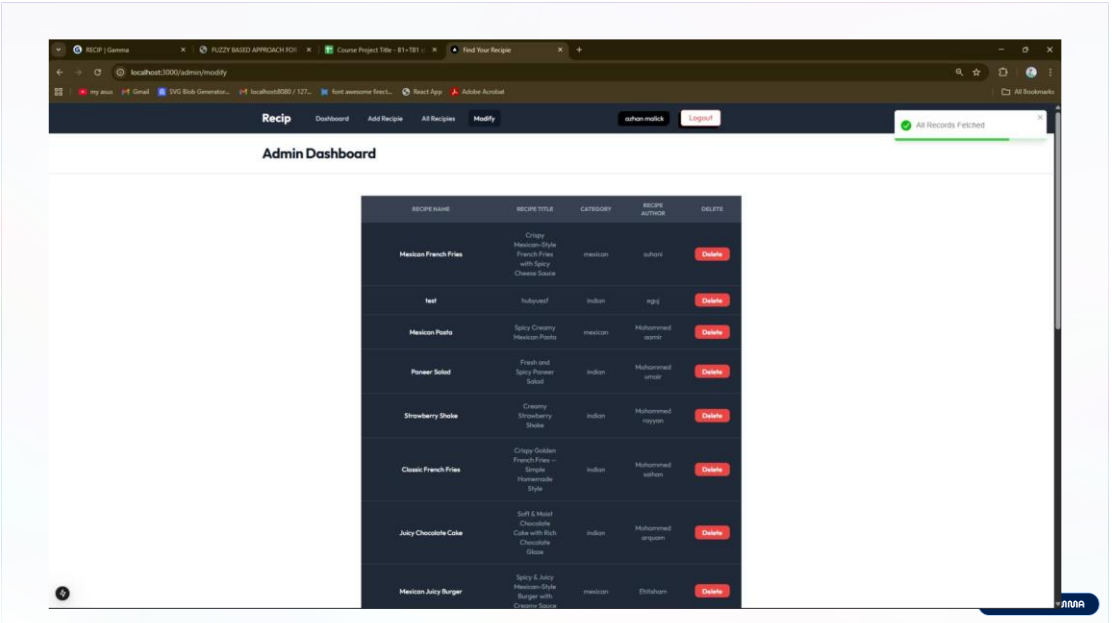

Recip
Sign up
cilantro 2 tablespoons mint leaves 2 tablespoons lemon juice 2 tablespoons ghee (clarified butter) Other: 2 tablespoons rice flour (optional, for consistency) Preparation: 1. Prepare the Meat (Mutton/Chicken): Wash and clean the mutton (or chicken) pieces. In a large pot, add the meat, ginger-garlic paste, chopped onions, green chilies, turmeric powder, red chili powder, and salt. Pour water (about 2-3 cups) and cook the meat until tender. You can use a pressure cooker for faster cooking (about 20-30 minutes). Once cooked, shred the meat pieces and set them aside. You can also blend the meat with some stock to get a smooth texture. 2. Prepare the Lentils and Grains: Wash the wheat and lentils thoroughly. In a separate large pot, add the wheat, moong dal, chana dal, toor dal, and masoor dal. Add enough water (about 4-5 cups) and cook the grains and lentils together. Cook until they are completely soft and mushy. You can use a pressure cooker for this step as well. Once the grains and lentils are cooked, blend them into a smooth consistency. You can use a hand blender or a regular blender. Some people like to keep it a little chunky, so blend according to your preference. 3. Fry the Spices: In a separate frying pan, heat 2-3 tablespoons of ghee. Add cumin seeds, bay leaves, cardamom, cinnamon, and cloves. Fry for a minute until the spices release their aroma. Add this tempered ghee and spices into the cooked lentils and wheat mixture. 4. Combine the Meat and Lentils: Add the shredded meat to the lentil and wheat mixture. Add garam masala, coriander powder, cumin powder, and black pepper powder. Mix everything thoroughly. If the mixture is too thick, add more water to adjust the consistency (Haleem should be a smooth, porridge-like consistency). Let it simmer on low heat for 30-40 minutes, stirring occasionally. The Haleem needs to be cooked and stirred well so that all the flavors blend together. 5. Final Touches: Once the Haleem is fully cooked and the consistency is perfect, check for seasoning and adjust salt and spices if needed. Add lemon juice to enhance the flavor. 6. Garnish and Serve: Serve the Hyderabadi Haleem hot, garnished with fried onions, chopped cilantro, mint leaves, and a drizzle of ghee. You can also serve it with a side of naan, paratha, or a slice of lemon for extra zest. Tips: Meat: Traditionally, mutton is used for Haleem, but chicken can be used as a substitute for a quicker cooking process. Consistency: Haleem is best when it's thick and creamy. Adjust the water based on how thick or thin you prefer it. Slow Cooking: The key to a great Haleem is slow cooking and stirring to bring out the flavors. Some people like to cook it for 3-4 hours, and it really enhances the flavor. Rice Flour: Adding a little rice flour helps thicken the mixture if it's too watery. Enjoy your delicious and rich Hyderabadi Haleem!
A Recipe By Ahmed Shah
Like The Recipe?
17
Copy recipe link
Follow Us


Welcome to Recip
Your Username
johndoe
Your Email
name@flowbite.com
Your Password
Confirm Password
Already have an account? Signin instead
Register


Recip
© 2025 Recip. All rights reserved.
Follow Us


Recip
Dashboard
Add Recipie
All Recipies
Modify
ashan malick
Logout
Admin Dashboard
Total No Of Recipies Found : 19
Recip
© 2025 Recip. All rights reserved.
Follow Us


Add New Recipe
This information will be displayed publicly.
Recipe Information
Recipe Name
Category
Indian
Recipe Title
Steps
Author
Email
Cover Photo
Upload a file
Cancel
Add Recipe


Recip
Dashboard
Add Recipie
All Recipies
Modify
ashan.malick
Logout
All Records Fetched
Admin Dashboard
RECIPE NAME
RECIPE TITLE
CATEGORY
RECIPE AUTHOR
DELETE
Mexican French Fries
Crispy Mexican-Style French Fries with Spicy Cheese Sauce
mexican
ashani
Delete
test
hubyuesf
indian
egyj
Delete
Mexican Pasta
Spicy Creamy Mexican Pasta
mexican
Mohammed aamir
Delete
Paneer Salad
Fresh and Spicy Paneer Salad
indian
Mohammed umair
Delete
Strawberry Shake
Creamy Strawberry Shake
indian
Mohammed nayyan
Delete
Classic French Fries
Crispy Golden French Fries – Simple Homemade Style
indian
Mohammed salman
Delete
Juicy Chocolate Cake
Soft & Moist Chocolate Cake with Rich Chocolate Glaze
indian
Mohammed anquam
Delete
Mexican Juicy Burger
Spicy & Juicy Mexican-Style Burger with Creamy Sauce
mexican
Ehtisham
Delete